NATIONAL UNIVERSITY OF SINGAPORE
DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

ACADEMIC YEAR 2025-2026
SEMESTER 1

EE2213: Introduction to AI

TUTORIAL 3: L3 & L4

Question 1 (Lecture 3 & Lecture 4)

Which of the following search algorithms typically use a priority queue to implement the frontier? Select all that apply.

(a) Breadth-first search
(b) Uniform cost search (i.e., Dijkstra's algorithm)
(c) Greedy best-first search
(d) A* search

Ans: (b) (c) (d)

Question 2 (Lecture 4)

Which of the following is true about A* search? Select all that apply.

(a) The heuristic function can overestimate the true cost to the goal without affecting the optimality of A* search.
(b) A* search is generally complete and returns a solution if one exists, provided the search space is finite and all step costs are positive.
(c) Every consistent heuristic is also admissible.
(d) The graph-search of A* is optimal as long as the heuristic is admissible.

Ans: (b) (c)

Question 3 (Lecture 4)

Prove the following statement:

If a heuristic $h(n)$ is consistent, then it is also admissible.

Hint: If a heuristic is consistent, then for every node n and each of its successors n', $h(n) \leq c(n, n') + h(n')$ and $h(\text{goal}) = 0$. Here, $h(n)$ is the estimated distance from n to the goal (given by the heuristic). $c(n, n')$ is the actual distance from n to its neighbour n'. $h^*(n)$ is the true shortest distance between n and the goal. For induction assume that, $h(n') \leq h^*(n')$ for all successor n' of n. Then prove that, $h(n') \leq h^*(n')$ as well.

Ans: Prove by induction

Base case:

For the goal node $n = \text{goal}$,

By definition of consistency:

$$h(\text{goal}) = 0$$

$$h(\text{goal}) \leq h^*(\text{goal})$$

So the heuristic is admissible (doesn't overestimate).

Inductive step:

Assume that all nodes at most k steps away from the goal,

$$h(n) \leq h^*(n)$$

Now, let n be the predecessor of n' on that shortest path (so $n \to n' \to \cdots \to$goal), i.e., n is $k+1$ steps away from the goal along a shortest path.

By consistency:

$$h(n) \leq c(n, n') + h(n')$$

By the inductive hypothesis:

$$h(n') \leq h^*(n') \quad \text{(since } n' \text{ is closer to the goal)}$$

So:

$$h(n) \leq c(n, n') + h^*(n') = h^*(n)$$

Therefore, h is admissible at node n. Inductive step holds.

Question 4 (Lecture 4)

You are working on implementing A* search in a navigation app that finds the shortest path between two cities on a map. The graph is represented as an adjacency list, edge weights represent driving time, and each city has a precomputed heuristic value $h(n)$, which estimates the remaining travel time to the goal.

Below is a partially written pseudocode for the A* search algorithm. Some parts are missing. Please answer the following questions.

(i) Fill in the blanks (1) – (4) with the correct logic in the pseudocode.

(ii) Explain how the f-score is used to prioritize nodes in the open list.

(iii) The performance of A* heavily depends on the quality of the heuristic $h(n)$. What are the consequences if the heuristic overestimates the actual cost to the goal?

```
function A_Star(Graph, start, goal, h):
    for each node n in Graph:
        g[n] ← ________                          // (1) cost from start to n
        f[n] ← ________                          // (2) estimated total cost (f = g + h)
        prev[n] ← null

    g[start] ← 0
    f[start] ← h[start]
    open ← priority queue ordered by f-value
    Insert(open, start)

    closed ← empty set

    while open is not empty:
        current ← Extract-Min(open)

        if current == goal:
            return Reconstruct-Path(prev, goal)

        Add(closed, current)

        for each neighbor in neighbors of current:
            if neighbor in closed:
                continue

            tentative_g ← g[current] + cost(current, neighbor)

            if tentative_g < g[neighbor]:
                prev[neighbor] ← _____________          // (3)
                g[neighbor] ← tentative_g
                f[neighbor] ← _____________            // (4)

                if neighbor in open:
                    Update-Priority(open, neighbor)
                else:
                    Insert(open, neighbor)

    return failure
```

Ans:

(i) Fill in the blanks (1) – (4)

(1) ∞ // Initialize g[n] to infinity

(2) ∞ // Initialize f[n] to infinity

(3) current // Update the predecessor to current

(4) g[neighbor] + h[neighbor] // Update the f-score

(ii) Explain how the f-score is used to prioritize nodes in the open list.
The f-score is used to estimate the total cost of a path going through a node to the goal.

- A* always selects the node with the lowest f-score from the open list (via priority queue).

- This ensures that paths that appear promising (i.e. with low estimated total cost) are explored first.
- Mathematically:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the known cost from the start to n, and $h(n)$ is the heuristic estimate from n to the goal.

(iii) The performance of A* heavily depends on the quality of the heuristic $h(n)$. What are the consequences if the heuristic overestimates the actual cost to the goal?

If the heuristic overestimates the actual cost to the goal, A* is no longer guaranteed to find the optimal path. The algorithm may incorrectly dismiss the true shortest path because the overestimated heuristic makes it seem more expensive than it actually is. As a result, A* might explore a different path that looks better under the faulty estimate, leading to a valid solution — but not necessarily the shortest one.

Question 5 (Optional – For Exploration)

Extend the STRIPS planner from Lecture 4 to model a robot performing a multi-step coffee-making process.

Initial State:

- An empty coffee pot on the table
- Coffee powder available
- A mug on the table
- Milk in a milk jug on the table
- Unlimited water available from the tap

Goal:
The robot must prepare a mug containing coffee with milk.

To achieve this, the robot can perform the following five high-level actions:

- Pour water from the tap into the coffee pot
- Boil the water in the coffee pot
- Make coffee in the coffee pot by combining boiled water and coffee powder
- Pour coffee from the pot into the mug
- Add milk from the milk jug to the coffee in the mug

To keep things simple, the robot can perform actions without moving—there's no need to model movement. The robot already knows how to fetch water from the tap. It can directly access the tap water, coffee powder, and milk jug.

Your task is to define each of these actions using STRIPS-style action schemas, specifying

preconditions and effects. Then, implement a Breadth-First Search (BFS) planner to find a valid action sequence that leads the robot from the initial state to the goal.

Your solution should explore multiple intermediate states and must include at least all five of the listed actions in the final plan.

Ans: please refer to TUT3-Q5.py for sample implementation.